Week 3: Control Flow: Loops

Learning Outcome/Trait Assessment:

- Utilize for loops for iteration over sequences and ranges.
- Implement while loops for indefinite repetition based on a condition.
- Control loop execution using break and continue statements.
- Apply nested loops for complex iterative tasks.
- **Trait Assessment:** Efficient iteration, handling repetitive tasks, understanding loop termination conditions, preventing infinite loops, logical application of loop control statements.

Detailed Lecture Topics:

1. **The Need for Loops:**
    - Why do we need loops? (Automation, repetition, processing collections of data).
    - Contrast with manual repetition (demonstrate how tedious it is to print() 100 times).
2. **for Loop:**
    - Syntax: for item in sequence:.
    - Concept of iteration: repeating a block of code for each item in a sequence.
    - **The range() Function:**
        - range(stop): 0 up to (but not including) stop.
        - range(start, stop): start up to (but not including) stop.
        - range(start, stop, step): start up to stop, incrementing by step.
        - Common use cases for range() (e.g., repeating N times, counting).
    - Iterating over strings (character by character).
    - (Brief mention of iterating over lists/tuples – full coverage in Week 4).
3. **while Loop:**
    - Syntax: while condition:.
    - Concept: Repeats a block of code *as long as* a condition remains True.
    - Importance of initialization, condition, and update (to avoid infinite loops).
    - Examples: user input validation, simple counters.
4. **Loop Control Statements:**
    - **break:** Immediately terminates the current loop, execution continues after the loop.
    - **continue:** Skips the rest of the current iteration and moves to the next iteration of the loop.
    - Demonstrate clear examples for both.
5. **Nested Loops:**
    - Loops inside other loops.
    - How the inner loop completes all its iterations for each iteration of the outer loop.
    - Common applications: working with grids/tables, 2D structures, generating patterns.

Detailed Labs:

- **Lab 3.1: Multiplication Table Generator**
  - **Objective:** Students will use for loops (potentially nested) to generate a multiplication table for a given number or for numbers up to a certain range.
  - **Tasks (Common for Local & Colab):**
    1. Create a new Python file (lab3_1.py) or Colab code cell.
    2. **Part 1: Single Number Table:**
       - Prompt the user to enter a number (e.g., 7).
       - Use a for loop with range(1, 13) to print the multiplication table for that number from 1 to 12.
       - *Example Output for 7:*
         `7 x 1 = 7`
         `7 x 2 = 14`
         `...`
         `7 x 12 = 84`
    3. **Part 2: Full Multiplication Grid (Challenge/Extension):**
       - Use **nested for loops** to print a 12x12 multiplication grid.
       - Format the output neatly (e.g., using print(f"{i*j:4d}", end="") and print() after each row for spacing).
       - *Hint:* The outer loop iterates through rows, the inner loop iterates through columns.
- **Lab 3.2: Countdown & User Guessing Game**
  - **Objective:** Students will implement while loops for a countdown and a simple guessing game, incorporating break for early exit.
  - **Tasks (Common for Local & Colab):**
    1. Create a new Python file (lab3_2.py) or Colab code cell.
    2. **Part 1: Countdown Timer:**
       - Prompt the user to enter a starting number for the countdown.
       - Use a while loop to count down from that number to 0, printing each number.
       - After the loop, print "Blast off!"
    3. **Part 2: Simple Guessing Game:**
       - Choose a secret number (e.g., secret_number = 42).
       - Use a while True loop to continuously ask the user to guess the number.
       - If the guess is too low, print "Too low! Try again."
       - If the guess is too high, print "Too high! Try again."
       - If the guess is correct, print "Congratulations! You guessed it!" and use break to exit the loop.
       - **Challenge:** Add a limited number of attempts (e.g., 5 attempts) and use a break or else clause for the loop if attempts run out.

Detailed Activities:

- **Activity 3.1: Debugging Infinite Loops & Loop Logic Errors**
  - **Format:** Individual or pair work, followed by class discussion/demonstration.
  - **Instructions:** Provide students with several buggy code snippets containing:
    - An infinite while loop (missing update of loop control variable).
    - A for loop that doesn't produce the desired range (incorrect range() arguments).
    - A loop where break or continue is used incorrectly.
  - Students must identify the bug, explain why it happens, and fix the code.
  - **Purpose:** Develop critical debugging skills, understand common loop pitfalls.
  - **Sample Buggy Snippets (for Handout/Projection):**

```
Python
# Buggy Snippet A: Infinite Loop
# counter = 0
# while counter < 5:
#     print(counter)
#     # Missing counter increment!

# Buggy Snippet B: Incorrect Range
# for i in range(1, 5): # Goal: print 1, 2, 3, 4, 5
#     print(i)

# Buggy Snippet C: Misplaced Break
# for char in "hello":
#     if char == 'l':
#         print("Found 'l', stopping!")
#         break
#     print(char) # This will print 'h', 'e', 'l' then break
```

- **Activity 3.2: Pattern Printing with Nested Loops**
  - **Format:** Individual creative coding challenge.
  - **Instructions:** Ask students to write Python code using nested for loops to print simple ASCII art patterns.
  - **Examples:**
    1. A square of asterisks:

```
*****
*****
*****
*****
*****
```

    2. A right-angled triangle of asterisks:

```
*
```

```
**
***
****
*****
```

3. **Challenge:** An inverted right-angled triangle or a pyramid.

- **Purpose:** Reinforce understanding of nested loops and the relationship between loop variables and printed output. Encourages algorithmic thinking and problem-solving.

Sample Snippets for Class and Colab Ready

Here are Python code snippets that can be directly used for demonstration and practice in both VS Code (saved as .py files) and Google Colab (in code cells).

1. for Loop and range()

VS Code (for_loop_demo.py):

Python

```
# for_loop_demo.py

print("--- Using for loop with range() ---")
# Loop 5 times (0 to 4)
for i in range(5):
    print(f"Iteration {i}")

print("\n--- Counting from 1 to 10 ---")
for num in range(1, 11): # Start at 1, go up to (but not including) 11
    print(num)

print("\n--- Counting by 2s from 0 to 10 ---")
for even_num in range(0, 11, 2): # Start 0, end 11 (exclusive), step 2
    print(even_num)

print("\n--- Iterating over a string ---")
my_string = "Python"
for char in my_string:
```

```
    print(f"Character: {char}")

print("\n--- Calculating sum of first 5 numbers ---")
total_sum = 0
for i in range(1, 6): # Numbers 1, 2, 3, 4, 5
    total_sum += i # Equivalent to total_sum = total_sum + i
print(f"Sum of first 5 numbers: {total_sum}")
```

Google Colab (in a code cell):

```
    Python


# for_loop_demo in Colab

print("--- Using for loop with range() ---")
# Loop 5 times (0 to 4)
for i in range(5):
    print(f"Iteration {i}")

print("\n--- Counting from 1 to 10 ---")
for num in range(1, 11): # Start at 1, go up to (but not including) 11
    print(num)

print("\n--- Counting by 2s from 0 to 10 ---")
for even_num in range(0, 11, 2): # Start 0, end 11 (exclusive), step 2
    print(even_num)

print("\n--- Iterating over a string ---")
my_string = "Python"
for char in my_string:
    print(f"Character: {char}")

print("\n--- Calculating sum of first 5 numbers ---")
total_sum = 0
for i in range(1, 6): # Numbers 1, 2, 3, 4, 5
    total_sum += i # Equivalent to total_sum = total_sum + i
print(f"Sum of first 5 numbers: {total_sum}")
```

2. while Loop and Loop Control (break, continue)

VS Code (while_loop_control.py):

Python

```
# while_loop_control.py

print("--- Simple while loop (countdown) ---")
count = 5
while count > 0:
    print(f"Count: {count}")
    count -= 1 # Decrement count (important to avoid infinite loop)
print("Countdown finished!")

print("\n--- Input validation with while loop ---")
user_input = ""
while not (user_input == "yes" or user_input == "no"):
    user_input = input("Please enter 'yes' or 'no': ").lower()
print(f"You entered: {user_input}")

print("\n--- Using 'break' to exit early ---")
secret_number = 7
attempts = 0
max_attempts = 3
while attempts < max_attempts:
    guess_str = input(f"Guess the number (1-10) - Attempt {attempts + 1}/{max_attempts}: ")
    guess = int(guess_str)
    attempts += 1
    if guess == secret_number:
        print("Congratulations! You guessed it!")
        break # Exit loop immediately if correct
    elif guess < secret_number:
        print("Too low!")
    else:
        print("Too high!")
else: # This 'else' block runs if the loop completes *without* a break
    print(f"Sorry, you ran out of attempts. The secret number was {secret_number}.")
```

```
print("\n--- Using 'continue' to skip iterations ---")
for i in range(1, 11): # Numbers 1 to 10
    if i % 2 != 0: # If number is odd, skip the print
        continue # Go to the next iteration
    print(f"Even number: {i}") # Only even numbers will be printed
```

Google Colab (in a code cell):

Python

```
# while_loop_control in Colab

print("--- Simple while loop (countdown) ---")
count = 5
while count > 0:
    print(f"Count: {count}")
    count -= 1 # Decrement count (important to avoid infinite loop)
print("Countdown finished!")

print("\n--- Input validation with while loop ---")
user_input = ""
while not (user_input == "yes" or user_input == "no"):
    user_input = input("Please enter 'yes' or 'no': ").lower()
print(f"You entered: {user_input}")

print("\n--- Using 'break' to exit early ---")
secret_number = 7
attempts = 0
max_attempts = 3
while attempts < max_attempts:
    guess_str = input(f"Guess the number (1-10) - Attempt {attempts + 1}/{max_attempts}: ")
    guess = int(guess_str)
    attempts += 1
    if guess == secret_number:
        print("Congratulations! You guessed it!")
        break # Exit loop immediately if correct
    elif guess < secret_number:
```

```
        print("Too low!")
    else:
        print("Too high!")
else: # This 'else' block runs if the loop completes *without* a break
    print(f"Sorry, you ran out of attempts. The secret number was {secret_number}.")

print("\n--- Using 'continue' to skip iterations ---")
for i in range(1, 11): # Numbers 1 to 10
    if i % 2 != 0: # If number is odd, skip the print
        continue # Go to the next iteration
    print(f"Even number: {i}") # Only even numbers will be printed
```

3. Nested Loops (Pattern Printing Example)

VS Code (nested_loops_patterns.py):

Python

```
# nested_loops_patterns.py

print("--- Printing a 5x5 square of asterisks ---")
size = 5
for row in range(size):
    for col in range(size):
        print("* ", end="") # Print asterisk and space, don't go to new line
    print() # Go to the next line after each row is complete

print("\n--- Printing a right-angled triangle ---")
height = 5
for row in range(1, height + 1): # For each row (1 to 5)
    for col in range(row): # Print 'row' number of asterisks
        print("* ", end="")
    print()

print("\n--- Printing an inverted right-angled triangle ---")
for row in range(height, 0, -1): # From 5 down to 1
    for col in range(row):
        print("* ", end="")
```

```
    print()
```

Google Colab (in a code cell):

```
    Python
```

```
# nested_loops_patterns in Colab

print("--- Printing a 5x5 square of asterisks ---")
size = 5
for row in range(size):
    for col in range(size):
        print("* ", end="") # Print asterisk and space, don't go to new line
    print() # Go to the next line after each row is complete

print("\n--- Printing a right-angled triangle ---")
height = 5
for row in range(1, height + 1): # For each row (1 to 5)
    for col in range(row): # Print 'row' number of asterisks
        print("* ", end="")
    print()

print("\n--- Printing an inverted right-angled triangle ---")
for row in range(height, 0, -1): # From 5 down to 1
    for col in range(row):
        print("* ", end="")
    print()
```